Rising Waters

AI Flood Prediction System

IBM SmartBridge Internship Program

Project Overview

Floods are among the most destructive natural disasters, causing loss of life, damage to infrastructure, and severe economic disruption every year. Traditional flood forecasting methods are often slow and imprecise, which limits how quickly authorities can warn people and take preventive action.

Rising Waters is an AI-powered Flood Prediction System built using Machine Learning. It predicts flood probability from environmental and geographical parameters such as rainfall intensity, drainage capacity, climate conditions, urbanization level, and infrastructure quality. By learning patterns from historical flood data, the model can flag high-risk conditions before they turn into an emergency.

The trained model is integrated into a Flask web application, so users simply enter a few environmental parameters and instantly receive a flood risk prediction along with clear, AI-generated safety recommendations — no technical expertise required.

Objectives

The core aim of Rising Waters is to close the gap between raw environmental data and timely, actionable flood warnings. Conventional forecasting relies heavily on manual analysis and generalized rainfall thresholds, which often miss the local, compounding factors that actually decide whether an area floods — drainage capacity, soil saturation, and the level of urban development. This project sets out to replace that guesswork with a data-driven, learned model that adapts to real conditions on the ground. Beyond raw accuracy, the project is also about usability: a prediction system is only useful if the people who need it — disaster management officials, local authorities, and even everyday citizens — can access it quickly and understand the result without needing a background in data science. That is why the objectives below span the full journey from model design to deployment.

- Predict flood probability accurately using Machine Learning trained on real environmental data.
- Compare multiple ML algorithms — Decision Tree, Random Forest, KNN, and XGBoost — and select the best-performing model based on measurable evaluation metrics.
- Develop an easy-to-use, intuitive web application using Flask that anyone can operate without technical training.
- Support disaster management teams with early flood risk assessment so resources and warnings can be deployed before a crisis unfolds.

- Create a scalable solution that is ready for future cloud deployment, allowing the system to serve larger regions and higher traffic as adoption grows.

Features

Rising Waters is built around a small set of features that work together to turn raw environmental inputs into a clear, trustworthy risk signal. At its core is the prediction engine, powered by XGBoost, which was selected after being benchmarked directly against three other algorithms so that the final choice is backed by evidence rather than assumption. Around that engine sits a lightweight Flask application designed for speed — predictions return in real time, which matters when the whole point of the system is early warning. The output is never just a raw probability number; it is translated into an easy-to-read risk category and paired with an AI-generated safety recommendation, so the person reading the result immediately understands both the danger level and what to do next. Finally, the entire system was designed with growth in mind, following an architecture pattern that is compatible with IBM Cloud so it can move from a classroom prototype to a production-grade service without being rebuilt from scratch.

- Flood probability prediction from real-time environmental inputs
- AI-powered prediction engine using XGBoost
- Side-by-side comparison of multiple Machine Learning models
- Lightweight, responsive Flask web application
- Automatic flood risk level classification (Low / Moderate / High / Severe)
- AI-generated safety recommendations tailored to the risk level
- Fast, low-latency predictions suitable for real-time use
- IBM Cloud ready architecture for easy scaling

Technologies Used

The technology stack behind Rising Waters was chosen for reliability, speed of development, and how well each piece is documented and supported within the Python data science ecosystem. Python serves as the core programming language, since it offers direct access to mature machine learning and data-processing libraries without requiring custom tooling. Data handling is done with Pandas and NumPy, which together make it straightforward to clean, reshape, and engineer features from the raw flood dataset before it ever reaches a model. For the modelling layer, Scikit-learn provides the Decision Tree, Random Forest, and KNN implementations used during comparison, while XGBoost supplies the gradient-boosted model that ultimately became the production choice. Once a model is trained, Joblib is used to serialize and persist it to disk, so the exact trained model — not a retrained approximation — is what actually powers live predictions. Flask ties everything together as the web framework, handling incoming requests, routing form data to the model, and rendering results back to the user. On the frontend, plain HTML and CSS keep the interface simple, fast to load, and easy to customize. Finally, Git and GitHub

are used for version control, giving the team a clear history of changes and a shared, collaborative codebase throughout development.

Programming Language: Python

Libraries: Pandas, NumPy, Scikit-learn, XGBoost, Joblib, Flask

Frontend: HTML, CSS

Version Control: Git, GitHub

Machine Learning Workflow

Building a reliable prediction system required a disciplined, repeatable workflow rather than a one-off experiment. The process below covers every stage the project moved through, starting with raw data and ending with a live, deployable interface. Each stage was treated as a checkpoint: preprocessing was not considered complete until the dataset was verified clean, and no model was considered a candidate for deployment until it had been evaluated against the same held-out test data as every other model. This consistency is what allows the final comparison between Decision Tree, Random Forest, KNN, and XGBoost to be treated as fair and meaningful, rather than an artifact of different setups. The diagrams below illustrate both the step-by-step pipeline and the resulting system architecture that delivers predictions to the end user.

- Import required libraries
- Read and load the flood dataset
- Data preprocessing and cleaning
- Train-test split of the dataset
- Model training across multiple algorithms
- Model evaluation and comparison
- Save the best-performing model
- Build the Flask web application
- Deploy the prediction interface for end users

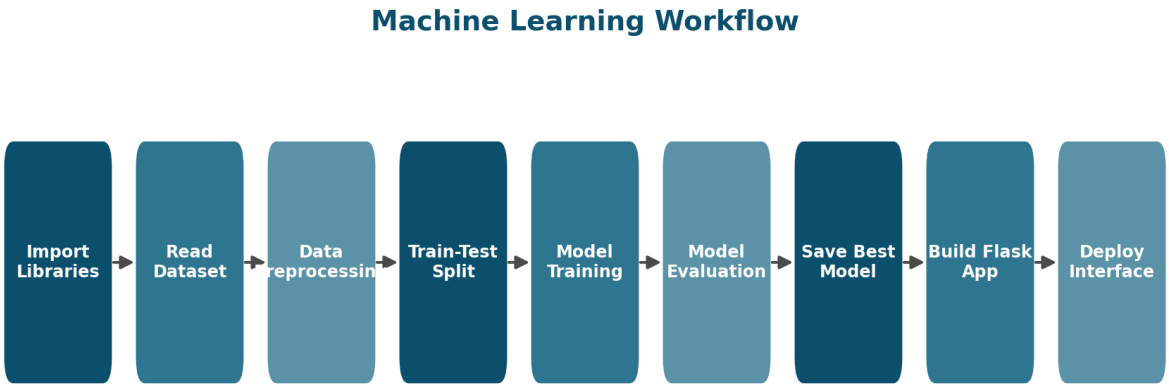


Figure 1: End-to-end machine learning workflow used in this project.

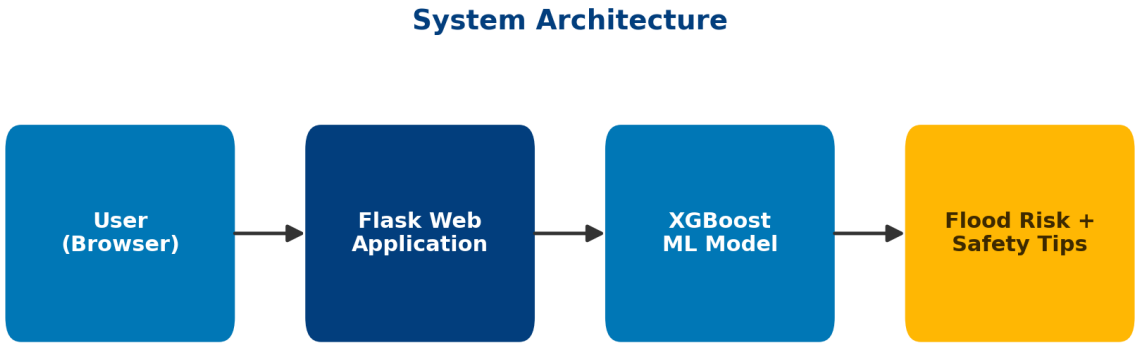


Figure 2: System architecture — from user input to AI-generated flood risk output.

Machine Learning Models

Choosing the right algorithm was treated as an empirical question, not a default decision. Four different approaches were trained on the same preprocessed flood dataset and evaluated under identical conditions so the results could be compared fairly. Decision Tree was included first as an interpretable baseline — it is fast to train and easy to explain, but tends to overfit and miss subtler patterns in the data. Random Forest builds on that idea by training many trees and combining their votes, which meaningfully improves accuracy and stability at the cost of some interpretability. K-Nearest Neighbours took a completely different approach, classifying new data points based on how similar they are to past examples, which performed well but scales less efficiently as the dataset grows. Finally, XGBoost — a gradient-boosted ensemble method — was trained with tuned hyperparameters and consistently outperformed the other three across accuracy, precision, and recall.

- Decision Tree — simple, interpretable baseline model
- Random Forest — ensemble of decision trees for improved accuracy
- K-Nearest Neighbours (KNN) — distance-based classification
- XGBoost — gradient-boosted ensemble, tuned for maximum accuracy

Among all the models tested, **XGBoost** achieved the best performance across accuracy, precision, and recall, and was selected as the final production model.

Model Performance

The table and chart below summarize how each model performed relative to the others. Performance was judged not just on raw accuracy but on how consistently each model handled edge cases in the data — situations with unusually heavy rainfall paired with strong drainage infrastructure, for example, where a naive model might overpredict risk. XGBoost's boosting approach, which corrects the errors of previous rounds of training, is what gave it the edge in exactly these harder cases.

Model	Performance
Decision Tree	Baseline Model

Random Forest	Good Performance
KNN	Better Performance
XGBoost	**Best Performing Model**

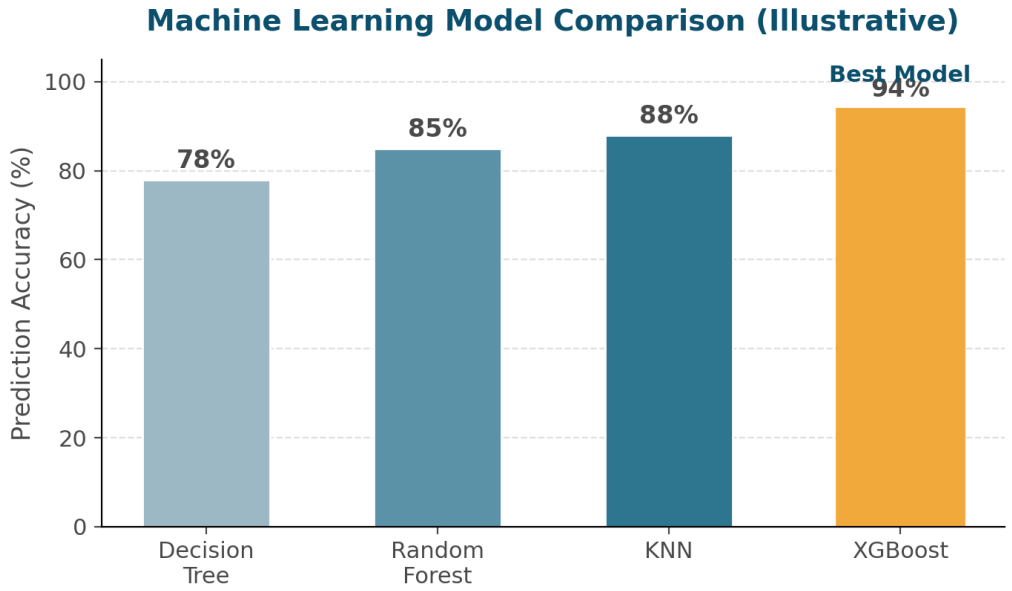


Figure 3: Illustrative accuracy comparison — XGBoost performs best.

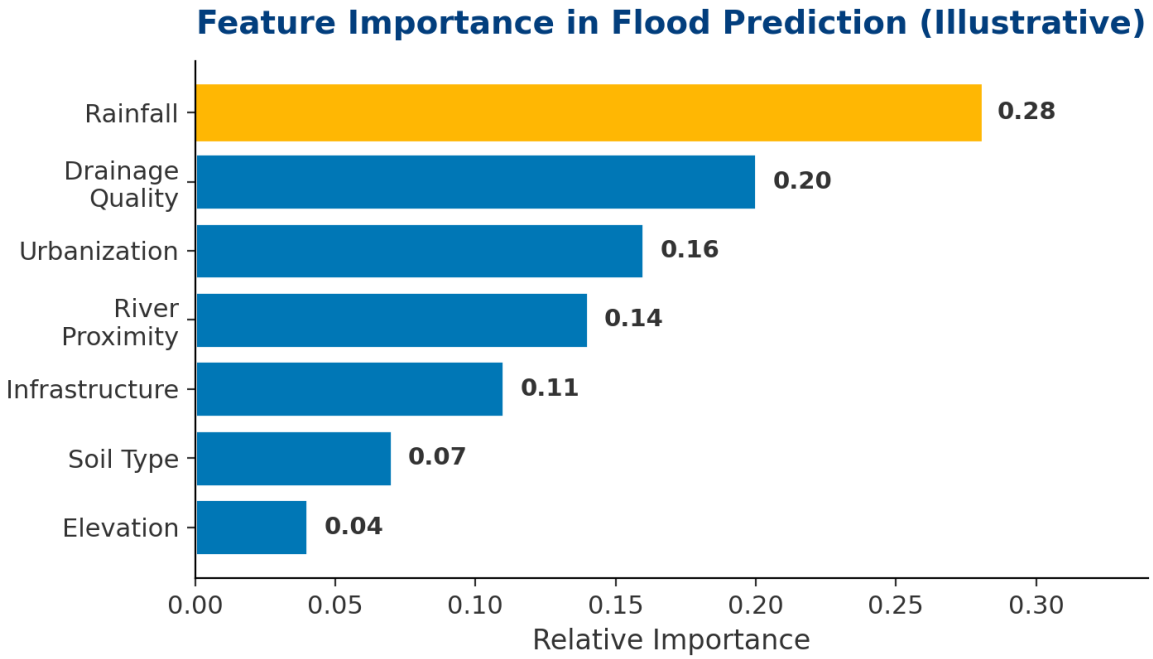


Figure 4: Illustrative feature importance — rainfall and drainage quality drive predictions most.

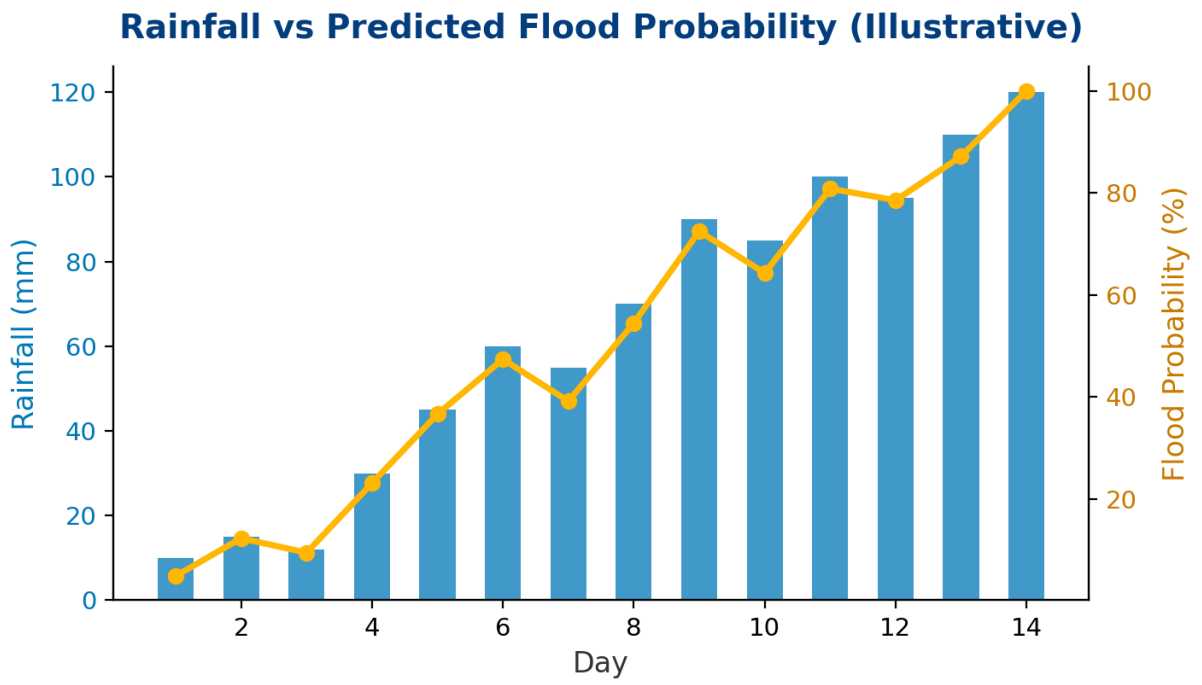


Figure 5: Illustrative relationship between rainfall and predicted flood probability over time.

Web Application

The Flask web application is the point of contact between the trained model and the people who need its output. It was deliberately kept simple: a single form asks for the relevant environmental parameters — rainfall, drainage quality, urbanization level, and related geographic factors — and submits them directly to the loaded XGBoost model in the backend. There is no unnecessary setup step and no account requirement, which matters in a genuine emergency context where every extra click costs time. Once the model returns a probability score, the application does not stop there; it converts that raw number into a clearly labeled risk category, so a non-technical user immediately understands whether the situation is low, moderate, high, or severe risk. Alongside the risk level, the app surfaces AI-generated safety recommendations appropriate to that category, turning a prediction into practical guidance rather than just a statistic on a screen.

- Enter environmental parameters through a simple form
- Predict flood probability using the trained XGBoost model
- View the classified flood risk level instantly
- Receive AI-based safety recommendations tailored to the risk level

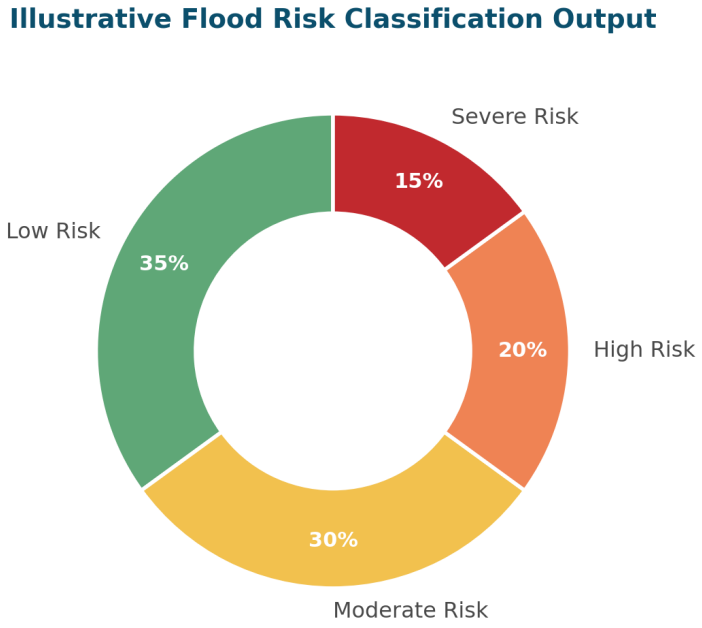


Figure 6: Illustrative flood risk category distribution shown to users.

Use Cases

Beyond the technical build, Rising Waters was designed with specific real-world users in mind, each of whom interacts with the system a little differently. Local authorities are the primary audience during an active weather event, using the tool to decide whether and when to issue evacuation notices. Disaster management organizations use it during the planning phase, allocating limited personnel and equipment toward the areas the model flags as highest risk rather than spreading resources evenly and reactively. Researchers and government agencies represent a third audience entirely, one that is less concerned with immediate action and more focused on using the model as a lens to re-examine historical flood records and validate whether existing risk assumptions still hold. Together, these three use cases show that the same underlying prediction engine can serve very different moments in the flood-response timeline — before, during, and after an event.

- **Early Flood Warning** — Authorities can monitor rainfall and environmental conditions to issue evacuation alerts before flooding occurs.
- **Disaster Resource Planning** — Disaster management teams can prioritize emergency response based on predicted flood risk.
- **Model Validation** — Government agencies and researchers can validate historical flood data using the trained machine learning model.

Future Scope

The current version of Rising Waters proves the core concept — that flood risk can be predicted reliably from environmental inputs and delivered through a simple web interface — but there is a clear roadmap for turning it into a production-grade public safety tool. The most immediate next step is connecting to a live weather API so

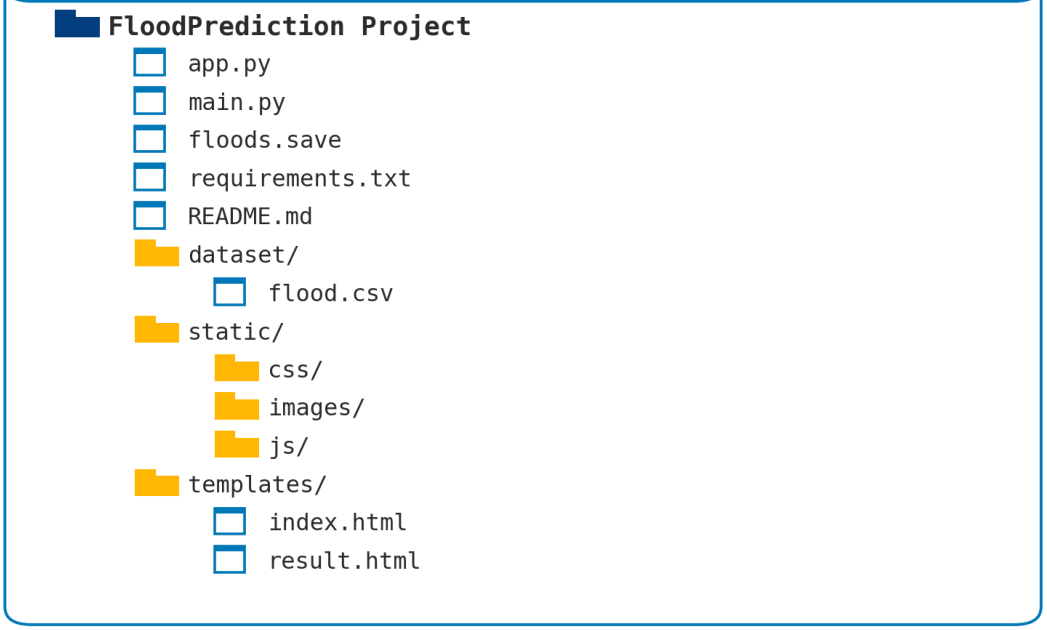
rainfall and climate data feed into the model automatically instead of being entered manually, removing a dependency on the user knowing current conditions. Location intelligence is another priority: Google Maps integration would let the system show risk visually across a region rather than as a single number, making it far easier for authorities to spot geographic clusters of danger. On the infrastructure side, a full IBM Cloud deployment would let the system handle far more concurrent users and scale automatically during major weather events, which is exactly when traffic would spike. A native mobile application and SMS or email alerting would extend reach to people without regular internet access, while a conversational AI assistant could help users interpret results and ask follow-up questions in plain language. Finally, a real-time public dashboard would let both officials and residents track risk across an entire region at a glance, closing the loop between prediction and community awareness.

- Live Weather API integration for real-time rainfall data
- Google Maps integration for location-based risk mapping
- Full IBM Cloud deployment for scalability
- Native mobile application for on-the-go alerts
- AI chat assistant for interactive flood guidance
- SMS and email alert notifications
- Real-time flood dashboard for authorities and the public

Project Structure

The codebase follows a conventional, easy-to-navigate Flask project layout so that anyone reviewing or extending the repository can quickly find what they need. Application logic lives in app.py and main.py at the root level, alongside the serialized model file, floods.save, which is loaded at runtime rather than retrained on every request. The dataset directory holds the raw flood.csv data used for training and evaluation, kept separate from application code so it can be swapped or updated independently. Static assets — CSS stylesheets, images, and JavaScript — are organized under the static folder, while the templates directory holds the HTML pages that Flask renders for the user, including the input form and the results view. This separation of concerns keeps the project maintainable as new features are added.

Project Directory Structure



Team Members

Rising Waters was built by a five-member team as part of the IBM SmartBridge Internship Program, with responsibilities spanning data preprocessing, model training and evaluation, Flask application development, and frontend design. Working across these areas together gave the team hands-on experience with the full lifecycle of a machine learning product — not just building a model in isolation, but shipping it as something a real user could actually open in a browser and interact with. The table below lists each member and their role on the project.

Name	Role
Shaik Sheema	Team Lead
Shaik Neha Masrath	Team Member
Salkapuram Samiya Mahima Jyothi	Team Member
Shaik Sadika	Team Member
Yesra Ameena	Team Member

Internship

This project was developed as part of the IBM SmartBridge Internship Program, a hands-on program that pairs students with real-world, industry-relevant problems and mentors them through the full build process — from initial data exploration to a deployed, working application. The internship structure emphasized practical skill-building over theory alone, requiring the team to justify every technical decision, from choice of algorithm to framework, with evidence rather than convention.

Thank You

Thank you for exploring our project.

Together, technology and AI can help build safer, more resilient communities.

Stay Safe • Predict Early • Save Lives